

Modeling Tabular and Structured Data

Logistic regression, regularization, forests, and boosting

Detailed lecture notes with scratch-versus-library evidence

MATH 550 Mathematical Data Science Capstone

Central lesson. A high-level estimator is easier to trust after we can trace its governing objective into primitive code, test the resulting behavior, and explain any remaining gap. Agreement in ROC AUC verifies ranking; it does not automatically verify probability calibration, optimization, or parameter equivalence.

Contents

1	Learning objectives and module map	4
2	The empirical question and public benchmark	4
2.1	Dataset and target	4
2.2	A fixed, leakage-resistant protocol	5
2.3	Why the comparison is paired	5
3	Evaluation: one metric cannot verify a model	5
3.1	Ranking, thresholds, and probabilities	5
3.2	What a scratch-versus-library gap can mean	6
4	Logistic regression from probability to code	6
4.1	Linear score and sigmoid	6
4.2	Bernoulli likelihood and cross-entropy	7
4.3	Gradient and curvature	7
4.4	Reusable estimator interface	7
5	Regularization and explicit optimization	8
5.1	L2 shrinkage	8
5.2	L1 sparsity and the proximal update	8
5.3	Acceleration, restart, and monotone acceptance	8
5.4	Matching scikit-learn's penalty convention	8
6	Decision trees and CART: the reusable tree primitive	9
6.1	What a decision tree represents	9
6.2	What CART means	10
6.3	Node impurity for classification	11
6.4	The CART split objective	11

6.5	A six-observation split calculation	12
6.6	Leaf predictions and probability caution	12
6.7	Stopping, pre-pruning, and cost-complexity pruning	12
6.8	Regression CART and the connection to boosting	13
6.9	What trees gain and what they sacrifice	13
7	Random forests: bagging plus randomized features	13
7.1	Why one tree is not enough	13
7.2	The random-forest algorithm	14
7.3	How averaging reduces variance	14
7.4	Out-of-bag evaluation	15
7.5	Hyperparameters and their effects	15
7.6	Interpretation and common traps	15
8	AdaBoost: sequential focus on difficult rows	16
8.1	Exponential loss and the weighted training sample	16
8.2	Deriving the weak-learner coefficient	17
8.3	Deriving the observation-weight update	17
9	Gradient boosting: descent in function space	18
9.1	From ordinary gradient descent to a functional gradient	18
9.2	Binary cross-entropy pseudo-residuals	19
9.3	Deriving the initial constant score	19
9.4	Why a regression-tree leaf stores a mean pseudo-residual	20
9.5	Optional terminal-region optimization and the bridge to Newton boosting	20
10	XGBoost-style second-order trees	21
10.1	Step 1: write the regularized stagewise objective	21
10.2	Step 2: take a second-order Taylor expansion	22
10.3	Step 3: group observations by tree leaf	22
10.4	Step 4: derive the optimal leaf value	23
10.5	Step 5: derive the split-gain formula	23
10.6	Interpretation and relation to Newton's method	24
11	LightGBM-style histograms and leaf-wise growth	24
12	Measured scratch-versus-library results	25
12.1	Held-out ranking consistency	25
12.2	Probability discrepancies are the main teaching result	26

12.3 The unregularized separation warning	27
12.4 Parameter verification	27
13 How to interpret model choice responsibly	27
14 Common failure modes and diagnostic questions	28
15 Extensions	28
A Pseudocode	29
A.1 Accelerated proximal logistic training	29
A.2 Greedy CART classification tree	29
A.3 Random forest	29
A.4 Newton boosting	30
B Reproducibility commands	30

1 Learning objectives and module map

By the end of the module, students should be able to:

1. derive binary cross-entropy from a Bernoulli likelihood and implement it with numerically stable formulas;
2. derive logistic gradients and connect L1/L2 penalties to proximal and smooth optimization updates;
3. explain and implement CART splitting, bootstrap aggregation, feature subsampling, and adaptive observation reweighting;
4. interpret gradient boosting as descent in function space and derive the second-order XGBoost leaf and split equations;
5. explain histogram split search and leaf-wise growth in LightGBM-style training;
6. conduct a controlled scratch-versus-library comparison on untouched test data using ranking, threshold, probability, and uncertainty metrics;
7. diagnose discrepancies using mathematical and software mechanisms rather than declaring one implementation “wrong” from a single metric.

The recommended classroom sequence is eight 75-minute meetings. Meetings 1–3 develop logistic regression and regularization; 4–5 develop trees, forests, and AdaBoost; 6–7 develop first- and second-order boosting; meeting 8 is an evidence lab using the benchmark outputs. Short code-reading exercises use the modules listed in Table 1.

File	Teaching responsibility
<code>models/linear.py</code>	score, sigmoid, cross-entropy, penalties, accelerated proximal-gradient training, and fitted interfaces
<code>models/trees.py</code>	weighted CART classification, regression trees, and Newton tree nodes
<code>models/ensembles.py</code>	bootstrap/random-feature forests and weighted-stump AdaBoost
<code>models/boosting.py</code>	first-order gradient boosting, exact Newton boosting, and histogram leaf-wise boosting
<code>evaluation.py</code>	binary metrics, probability checks, paired bootstrap, and agreement diagnostics
<code>experiments/tabular</code>	fixed configuration, controlled model pairs, diagnostics, artifact writing, and report generation

Table 1: The package is split by mathematical responsibility, not by arbitrary software layers.

2 The empirical question and public benchmark

2.1 Dataset and target

The classroom benchmark is the Breast Cancer Wisconsin (Diagnostic) dataset from the UCI Machine Learning Repository [1]. It has 569 rows and 30 continuous features computed from digitized images of a fine needle aspirate of a breast mass. The features summarize radius, texture, perimeter, area, smoothness, compactness, concavity, concave points, symmetry, and fractal dimension. There are no missing values in the bundled copy.

The original labels are malignant and benign. The module explicitly encodes

$$y_i = \begin{cases} 1, & \text{malignant,} \\ 0, & \text{benign.} \end{cases}$$

This direction matters: sensitivity is then the fraction of malignant cases correctly flagged, and the predicted probability is interpreted as the probability of malignancy. The data are licensed CC BY 4.0 by UCI. The official source files are preserved under `data/tabular/wdbc/`, so a benchmark run does not depend on a live download.

2.2 A fixed, leakage-resistant protocol

The experiment uses a class-stratified 80/20 split with random seed 550: 455 training observations and 114 test observations. The split is fixed before model fitting. Logistic predictors are standardized with means and standard deviations learned on the training set; tree models use original feature scales. Any regularization-path comparison uses only a further training/validation split inside the 455 training rows. The final test set is not a tuning resource.

This design supports a narrow claim: under the documented split and hyperparameters, does each compact implementation reproduce the relevant predictive behavior of its production counterpart? It does not establish a clinical decision rule, population transportability, or superiority among model families.

2.3 Why the comparison is paired

For each family, the scratch and library estimators see the same rows, labels, preprocessing policy, and nominal hyperparameters. Test AUC differences use 2,000 class-stratified paired-bootstrap samples. A bootstrap replicate samples positive and negative test indices with replacement, then evaluates both implementations on exactly the same sampled rows. Pairing removes resampling variation that would arise if the two models were evaluated on different replicates.

The pre-declared consistency rule is intentionally practical: a pair is called performance-consistent when the 95% paired-bootstrap interval for scratch minus library AUC contains zero, or the observed absolute AUC gap is no more than 0.02. This is a teaching tolerance, not a universal equivalence margin.

3 Evaluation: one metric cannot verify a model

Let $\hat{p}_i$ be the predicted probability for class 1 and $\hat{y}_i = \mathbb{I}(\hat{p}_i \geq 0.5)$.

3.1 Ranking, thresholds, and probabilities

- **ROC AUC** is the probability that a randomly selected positive receives a higher score than a randomly selected negative. It is invariant to strictly increasing transformations of the score.
- **Accuracy** is the fraction of correct labels at the chosen threshold. It can hide class imbalance.
- **Sensitivity** is $TP/(TP + FN)$ and **specificity** is $TN/(TN + FP)$.
- **Log loss** is

$$-\frac{1}{n} \sum_{i=1}^n \{y_i \log \hat{p}_i + (1 - y_i) \log(1 - \hat{p}_i)\}.$$

It strongly penalizes confident mistakes.

- **Brier score** is $n^{-1} \sum_i (y_i - \hat{p}_i)^2$. It evaluates probability accuracy on a quadratic scale.

AUC can agree perfectly while log loss differs: two models can induce the same ordering but map their margins to different probability ranges. Therefore the benchmark also reports probability mean absolute error (MAE), probability correlation, and label agreement between each implementation pair.

3.2 What a scratch-versus-library gap can mean

A gap is evidence to investigate, not a verdict. Common mechanisms include:

- different loss reduction conventions (sum versus mean);
- a different mapping between C and λ ;
- penalizing or not penalizing the intercept;
- different solvers, stopping rules, initialization, or line searches;
- different random-number streams, tie-breaking, or threshold candidates;
- optimized terminal-region values in a library boosting implementation;
- different score-to-probability transformations;
- production safeguards for missing data, numeric precision, or degenerate splits that are intentionally absent from a compact teaching implementation.

4 Logistic regression from probability to code

4.1 Linear score and sigmoid

For $x_i \in \mathbb{R}^p$, logistic regression forms the score

$$z_i = x_i^\top \beta + b$$

and applies the sigmoid

$$p_i = \sigma(z_i) = \frac{1}{1 + e^{-z_i}}.$$

The log-odds are linear:

$$\log \frac{p_i}{1 - p_i} = x_i^\top \beta + b.$$

The decision boundary at probability 0.5 is the hyperplane $x_i^\top \beta + b = 0$. Coefficients describe changes in log-odds for one-unit feature changes conditional on the other features; they are not automatically causal effects.

4.2 Bernoulli likelihood and cross-entropy

Assume conditionally independent Bernoulli responses:

$$p(y_i \mid x_i, \beta, b) = p_i^{y_i} (1 - p_i)^{1-y_i}.$$

The mean negative log-likelihood is

$$\begin{aligned} \ell(\beta, b) &= -\frac{1}{n} \sum_{i=1}^n [y_i \log p_i + (1 - y_i) \log(1 - p_i)] \\ &= \frac{1}{n} \sum_{i=1}^n [\log(1 + e^{z_i}) - y_i z_i]. \end{aligned}$$

The second form is the one implemented. Computing `logaddexp(0, z)` avoids overflow from explicitly evaluating e^z for a large positive score. A stable sigmoid uses separate algebra for positive and negative scores so that e^{-z} or e^z remains bounded.

4.3 Gradient and curvature

Because $\partial \ell_i / \partial z_i = p_i - y_i$, the gradient is

$$\nabla_{\beta} \ell = \frac{1}{n} X^{\top} (p - y), \quad \frac{\partial \ell}{\partial b} = \frac{1}{n} \mathbf{1}^{\top} (p - y).$$

Let $\tilde{X} = [\mathbf{1}, X]$ and $w = [b, \beta^{\top}]^{\top}$. The Hessian is

$$\nabla^2 \ell(w) = \frac{1}{n} \tilde{X}^{\top} W \tilde{X}, \quad W = \text{diag}\{p_i(1 - p_i)\}.$$

Since $0 \leq p_i(1 - p_i) \leq 1/4$, the gradient is Lipschitz with a global bound

$$L \leq \frac{\|\tilde{X}\|_2^2}{4n}.$$

The scratch optimizer uses $1/L$ as its initial step size. This is slower than a carefully tuned second-order method but makes the objective-to-update map visible.

4.4 Reusable estimator interface

`ScratchLogisticRegression` exposes `fit`, `decision_function`, `predict_proba`, and `predict`. It also exposes `coef_`, `intercept_`, `loss_curve_`, separate data and regularization loss histories, `n_iter_`, `converged_`, and the accepted step size. The public `objective_components` method lets a student verify

$$\text{objective} = \text{data loss} + \text{regularization loss}$$

at the fitted parameters. A finite-difference unit test independently verifies the analytic gradient.

5 Regularization and explicit optimization

5.1 L2 shrinkage

The L2 objective is

$$F(\beta, b) = \ell(\beta, b) + \frac{\lambda}{2} \|\beta\|_2^2.$$

The intercept is not penalized. The gradient contribution is $\lambda\beta$, and the smooth-gradient update is

$$\beta^{(k+1)} = \beta^{(k)} - \eta \left[\frac{1}{n} X^\top (p - y) + \lambda \beta^{(k)} \right].$$

L2 generally shrinks correlated coefficients together. It reduces variance and prevents coefficients from diverging under complete separation, but it rarely sets a coefficient exactly to zero.

5.2 L1 sparsity and the proximal update

The L1 objective is

$$F(\beta, b) = \ell(\beta, b) + \lambda \|\beta\|_1.$$

The absolute-value penalty is not differentiable at zero. A proximal-gradient step first takes a smooth gradient step $u = \beta - \eta \nabla \ell(\beta)$ and then applies soft thresholding coordinatewise:

$$\beta_j^+ = \text{sign}(u_j) \max(|u_j| - \eta\lambda, 0).$$

This is not a heuristic zeroing rule. It is the exact proximal operator of the scaled L1 norm. As λ increases, more coordinates cross the threshold and become exactly zero.

5.3 Acceleration, restart, and monotone acceptance

The implementation uses an accelerated proximal-gradient step in the style of FISTA. Acceleration extrapolates the next search point using the two most recent accepted iterates. Because extrapolation can overshoot, the implementation restarts momentum and halves the step until the penalized objective is no larger than the last accepted value. The recorded objective is therefore monotone up to floating-point tolerance.

This design serves two teaching goals: students can see a modern first-order method, and every accepted step remains easy to diagnose. Mature solvers may use coordinate descent, L-BFGS, trust regions, or screening rules and should be expected to run faster.

5.4 Matching scikit-learn's penalty convention

The scratch objective uses mean loss plus $\lambda\Omega(\beta)$. Scikit-learn's regularized logistic objective uses a convention equivalent to a summed data term with inverse strength C for the solvers used here. For $C = 1$ and n training observations, the matched scratch value is $\lambda = 1/n$. Forgetting this factor creates a genuine but avoidable parameter gap.

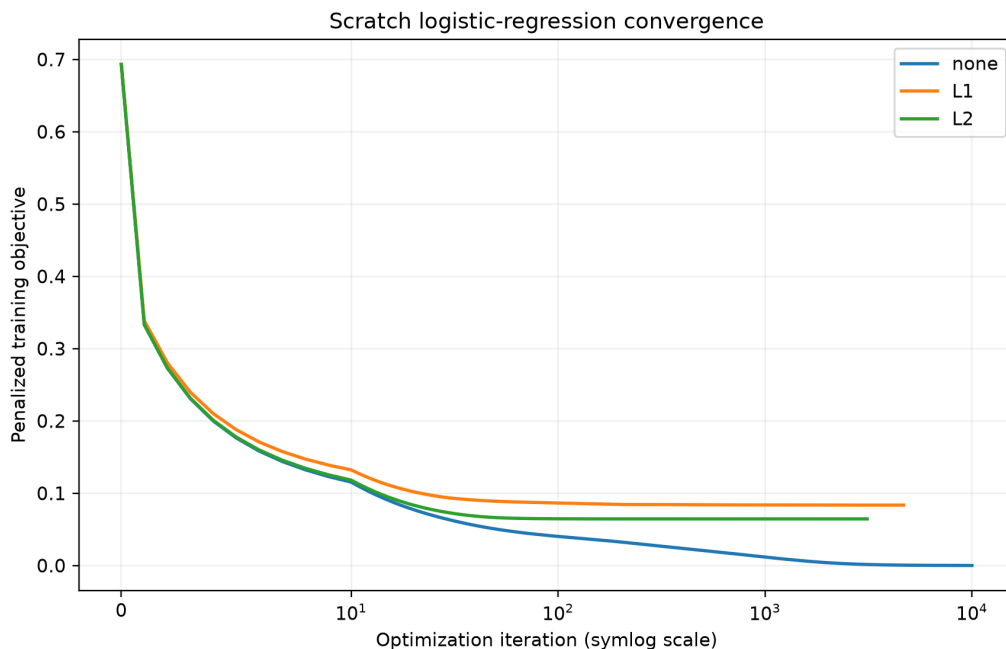


Figure 1: Accepted penalized-objective paths for the three scratch logistic models. The plot verifies optimization behavior, not held-out generalization.

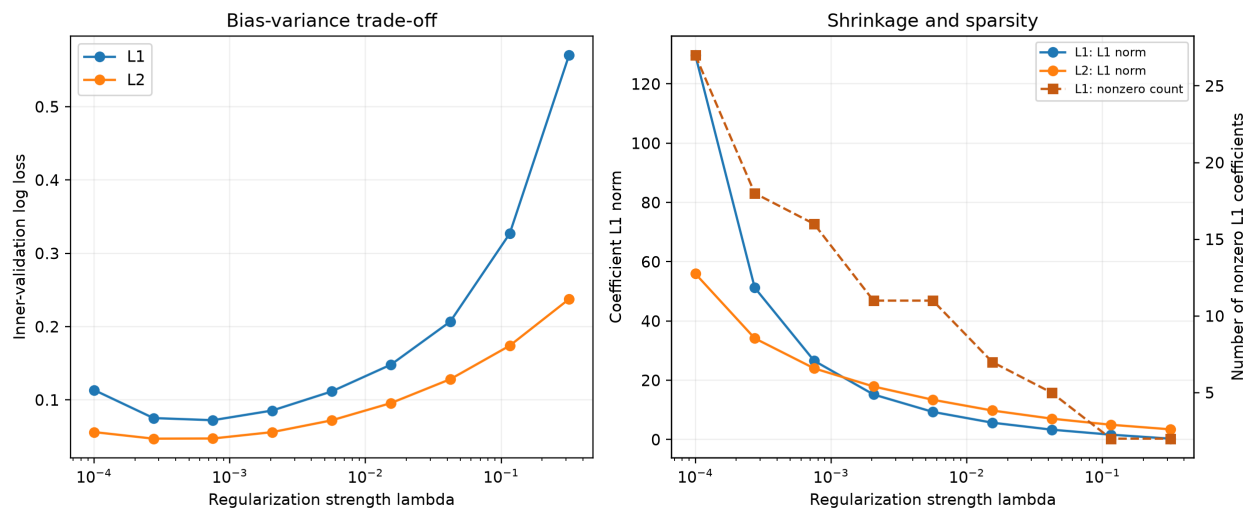


Figure 2: Training-only regularization study. Validation log loss, coefficient norm, and L1 support answer different questions and are all inspected before the test set is used.

6 Decision trees and CART: the reusable tree primitive

6.1 What a decision tree represents

A decision tree predicts by asking a sequence of simple questions. For a numeric feature, an internal node asks whether $x_j \leq t$. The answer sends the observation to the left or right child. A *root* is the first node,

an *internal node* contains a split, a *branch* is one answer to a split, and a *leaf* or *terminal node* stores the final prediction. The depth of a node is the number of splits between it and the root.

The resulting model is a recursive, axis-aligned partition of feature space. If the leaves define regions $R_1, \dots, R_L$, a binary classification tree has the piecewise-constant form

$$\hat{p}(x) = \sum_{\ell=1}^L \hat{p}_\ell \mathbb{I}(x \in R_\ell), \quad \hat{y}(x) = \mathbb{I}\{\hat{p}(x) \geq 1/2\}.$$

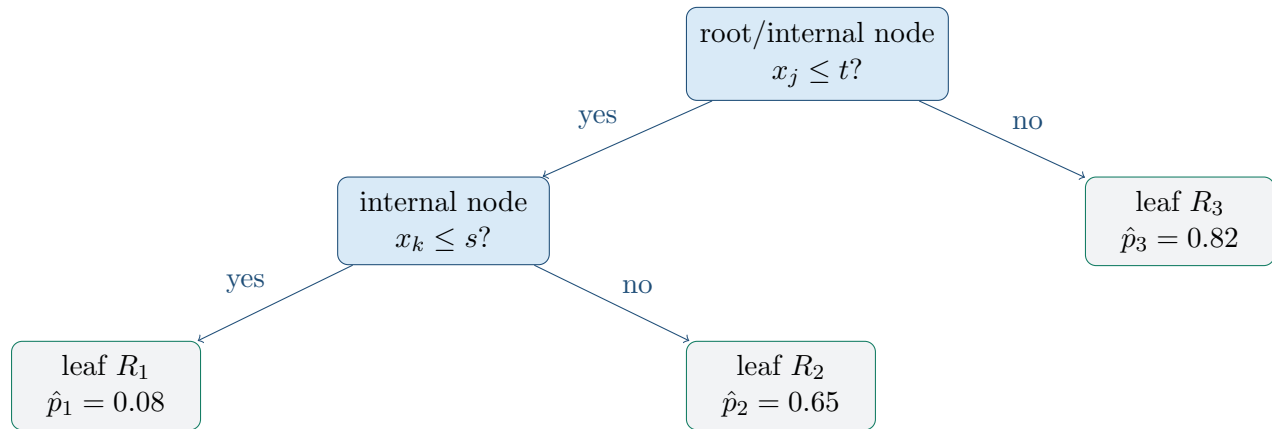


Figure 3: Tree anatomy and one prediction path. The symbols and leaf probabilities are illustrative rather than estimates from the benchmark fit.

Thus a tree can represent interactions without adding interaction columns by hand. A split on feature j may matter only inside a branch created by an earlier split on feature k . The price is discontinuity: two nearly identical observations can receive different predictions when they fall on opposite sides of a threshold.

Reading a prediction path. Start at the root, evaluate one question, follow the corresponding branch, and repeat until reaching a leaf. The conjunction of those branch conditions is a human-readable rule for that prediction. This local path is easier to explain than the full tree, whose number of leaves can grow exponentially with depth.

6.2 What CART means

CART stands for *Classification and Regression Trees* [2]. It uses binary recursive partitioning:

1. at the current node, search the allowed feature-threshold pairs;
2. choose the split that produces the largest immediate decrease in an impurity or loss criterion;
3. repeat the same procedure independently in each child;
4. stop or prune to control complexity.

This algorithm is *greedy*. It chooses the best next split, not the best complete tree over all possible future split sequences. A locally attractive root split can prevent a globally better tree from being reached. Exhaustive search over all trees is combinatorial, which is why the greedy approximation is central to practical tree learning.

6.3 Node impurity for classification

At a node $\mathcal{N}$ with nonnegative observation weights w_i , define total weight and weighted class proportions by

$$W = \sum_{i \in \mathcal{N}} w_i, \quad \hat{\pi}_k = \frac{1}{W} \sum_{i \in \mathcal{N}} w_i \mathbb{I}(y_i = k).$$

An impurity is small when one class dominates and large when classes are mixed. Three common choices are

$$\begin{aligned} I_{\text{error}}(\mathcal{N}) &= 1 - \max_k \hat{\pi}_k, \\ I_{\text{entropy}}(\mathcal{N}) &= - \sum_k \hat{\pi}_k \log \hat{\pi}_k, \\ I_{\text{Gini}}(\mathcal{N}) &= 1 - \sum_k \hat{\pi}_k^2. \end{aligned}$$

The convention $0 \log 0 = 0$ makes entropy well defined at a pure node. For binary classification with positive fraction p , Gini impurity simplifies to

$$I_{\text{Gini}} = 1 - p^2 - (1 - p)^2 = 2p(1 - p).$$

It is zero at $p = 0$ or 1 and maximized at $p = 1/2$. Misclassification error is useful for describing leaf error, but it is relatively insensitive to changes in class composition. Gini and entropy therefore usually provide a more informative split-search signal. The scratch classifier uses weighted Gini so the same tree primitive can later serve as an AdaBoost weak learner.

6.4 The CART split objective

For candidate split (j, t) , define

$$\mathcal{N}_L(j, t) = \{i \in \mathcal{N} : x_{ij} \leq t\}, \quad \mathcal{N}_R(j, t) = \{i \in \mathcal{N} : x_{ij} > t\}.$$

Let W_L and W_R be their total weights. CART chooses the split with minimum weighted child impurity

$$Q(j, t) = \frac{W_L}{W} I(\mathcal{N}_L) + \frac{W_R}{W} I(\mathcal{N}_R),$$

or, equivalently, maximum impurity decrease

$$\Delta I(j, t) = I(\mathcal{N}) - Q(j, t).$$

Weighting by child size prevents a tiny pure child from looking artificially competitive while leaving almost all observations in a mixed sibling. Subject to the minimum-leaf constraint, the implementation sorts the values of each candidate feature and considers thresholds midway between adjacent distinct values. No training partition changes between two such adjacent values, so searching other thresholds in that interval would be redundant.

After sorting, cumulative class weights provide the left and right impurities without rescanning every row for every threshold. Sorting one feature at a node costs $O(n_{\mathcal{N}} \log n_{\mathcal{N}})$; the subsequent threshold scan is linear in the number of rows at that node. Production libraries reuse ordering, parallelize features, or use histograms to reduce this cost.

6.5 A six-observation split calculation

Consider one feature and equally weighted binary labels:

x	1	2	3	4	5	6
y	0	0	0	1	1	1

The root has $p = 3/6 = 1/2$ and Gini impurity $2(1/2)(1/2) = 0.5$. At threshold $t = 3.5$, both children are pure, so

$$Q(1, 3.5) = \frac{3}{6}(0) + \frac{3}{6}(0) = 0, \quad \Delta I(1, 3.5) = 0.5.$$

At $t = 2.5$, the left child is pure, while the right child has positive fraction $3/4$ and Gini impurity $2(3/4)(1/4) = 0.375$. Therefore

$$Q(1, 2.5) = \frac{2}{6}(0) + \frac{4}{6}(0.375) = 0.25, \quad \Delta I(1, 2.5) = 0.25.$$

CART chooses 3.5 because its immediate impurity decrease is larger. In real data, ties can occur; deterministic feature order, floating-point tolerances, and random feature subsampling can then lead implementations to choose different but equally scored splits.

6.6 Leaf predictions and probability caution

The scratch classifier stores the weighted positive fraction in leaf ℓ ,

$$\hat{p}_{\ell} = \frac{\sum_{i \in R_{\ell}} w_i y_i}{\sum_{i \in R_{\ell}} w_i}.$$

This supplies both a class prediction and a probability interface. It does not guarantee calibrated probabilities. A leaf with two positive training examples returns probability one even though the estimate is extremely uncertain. Larger leaves, probability smoothing, calibration on separate data, or an ensemble can reduce this instability. Class weights change the quantity stored in a weighted leaf, so a weighted training proportion need not equal an unweighted population probability.

6.7 Stopping, pre-pruning, and cost-complexity pruning

If unconstrained, a classification tree can often grow until many leaves are pure. This lowers training impurity but raises variance. Common *pre-pruning* controls stop growth before that point:

- `max_depth`: cap the longest root-to-leaf path;
- `min_samples_split`: require enough rows to attempt a split;

- `min_samples_leaf`: require support on both sides of a split;
- `min_impurity_decrease`: require a meaningful local gain.

Classical CART also supports *post-pruning*. First grow a large tree T_0 , then select a subtree by minimizing the cost-complexity criterion

$$R_\alpha(T) = \sum_{\ell \in \mathcal{L}(T)} \frac{W_\ell}{W} I(\mathcal{N}_\ell) + \alpha |\mathcal{L}(T)|,$$

where $\mathcal{L}(T)$ is the leaf set. Increasing α trades training fit for fewer leaves. Weakest-link pruning generates a nested sequence of candidate subtrees; cross-validation can choose α using only training data. The course implementation deliberately uses pre-pruning rather than implementing the full pruning path. Scikit-learn exposes the analogous parameter as `ccp_alpha`.

6.8 Regression CART and the connection to boosting

For a regression leaf, squared-error loss is minimized by the mean response. For split (j, t) , regression CART minimizes

$$Q_{\text{SSE}}(j, t) = \sum_{i \in \mathcal{N}_L} (y_i - \bar{y}_L)^2 + \sum_{i \in \mathcal{N}_R} (y_i - \bar{y}_R)^2.$$

The reduction in sum of squared errors plays the same role as Gini decrease. First-order gradient boosting reuses this tree with pseudo-residuals in place of the original response. Second-order boosting instead uses sums of gradients and Hessians to define split gain and leaf values. Separate concise classes make these distinct leaf objectives visible.

6.9 What trees gain and what they sacrifice

Trees naturally model thresholds, nonlinearities, and conditional interactions; they require neither feature scaling nor a linear decision boundary. A single tree is also easy to trace locally. However, greedy fitting is unstable: a small change in the training rows can change the root split and therefore the entire subtree below it. Deep trees have low bias but high variance, while shallow or heavily pruned trees have higher bias but greater stability.

A monotone transformation preserves feature order. CART therefore produces the same candidate partitions after multiplying a feature by 1,000, although the reported threshold changes. Scaling can still matter through numeric precision, feature engineering, or other steps in a mixed pipeline. Missing values and categorical predictors require an explicit policy; the classroom data are finite and numeric, so the scratch tree intentionally omits imputation, surrogate splits, and native categorical subset search.

7 Random forests: bagging plus randomized features

7.1 Why one tree is not enough

The instability of an individual tree suggests averaging many versions trained on perturbed data. Bootstrap aggregation, or *bagging*, draws a bootstrap sample of n rows with replacement, fits one tree, repeats

independently, and averages predictions. For a given bootstrap sample, the expected fraction of distinct training rows is

$$1 - \left(1 - \frac{1}{n}\right)^n \rightarrow 1 - e^{-1} \approx 0.632.$$

Thus about 36.8% of rows are out of bag for a particular tree. Different bootstrap samples perturb the fitted tree and make averaging useful. However, ordinary bagged trees can remain highly correlated when the same strong feature dominates most root splits.

A random forest adds a second source of diversity: at *every node*, it draws a fresh subset of candidate features and searches for the best split only within that subset. This may weaken an individual tree, but it prevents a few strong predictors from making all trees look alike.

7.2 The random-forest algorithm

For tree $m = 1, \dots, M$:

1. draw n training indices with replacement;
2. start a CART tree on that bootstrap sample;
3. at each node, draw m_{try} of the p features without replacement;
4. find the best CART split among only those candidate features;
5. grow until the depth, leaf-size, purity, or gain rule stops the tree.

For the binary classroom implementation, tree m returns its leaf positive fraction $\hat{p}_m(x)$ and the forest averages

$$\hat{p}_{\text{RF}}(x) = \frac{1}{M} \sum_{m=1}^M \hat{p}_m(x), \quad \hat{y}_{\text{RF}}(x) = \mathbb{I}\{\hat{p}_{\text{RF}}(x) \geq 1/2\}.$$

Probability averaging is not always identical to majority voting over hard tree labels. The course implementation and its scikit-learn reference both use the average class-probability interface for the reported probability metrics.

7.3 How averaging reduces variance

Suppose the prediction errors of the trees have common variance σ^2 and pairwise correlation ρ . The variance of their average is approximately

$$\text{Var}\left(\frac{1}{M} \sum_{m=1}^M T_m(x)\right) = \rho\sigma^2 + \frac{1-\rho}{M}\sigma^2.$$

Increasing M shrinks the second term, but the correlated component remains. Bootstrap sampling creates different training sets; feature subsampling aims to reduce ρ . This equation explains two practical facts: adding trees cannot repair a strongly biased tree design, and decorrelating trees can matter more than merely adding highly similar trees.

7.4 Out-of-bag evaluation

For training row i , let $\mathcal{O}_i$ contain the trees whose bootstrap sample omitted that row. Its out-of-bag prediction is

$$\hat{p}_i^{\text{OOB}} = \frac{1}{|\mathcal{O}_i|} \sum_{m \in \mathcal{O}_i} \hat{p}_m(x_i).$$

Comparing these predictions with y_i gives an internal generalization estimate without fitting a separate validation forest. OOB predictions are available only from trees that did not train on the row, and with very few trees some rows may have too few OOB votes. Preprocessing performed before the bootstrap step can also leak information into an OOB estimate. The course benchmark therefore uses one untouched test set for every model pair; OOB error is a complementary forest diagnostic, not the source of the reported test metrics.

7.5 Hyperparameters and their effects

Control	Main effect and diagnostic question
Number of trees M	More trees stabilize the Monte Carlo average but increase time and memory. Has OOB or validation performance plateaued?
m_{try} / <code>max_features</code>	Smaller subsets reduce tree correlation but can weaken splits. Are the trees diverse without becoming systematically poor?
<code>max_depth</code>	Greater depth lowers individual-tree bias and raises variance and cost. Does held-out probability quality deteriorate?
<code>min_samples_leaf</code>	Larger leaves smooth probabilities and regularize local rules. Are tiny leaves producing extreme probabilities?
Bootstrap sample size	Smaller samples increase diversity but give each tree less information. Is the strength-correlation tradeoff favorable?
Class/sample weights	Can emphasize costly or rare cases, but changes the fitted target and probability interpretation. Are evaluation metrics aligned with that choice?

Table 2: Random-forest controls should be interpreted through tree strength, tree correlation, probability stability, and computational cost.

For classification, $m_{\text{try}} \approx \sqrt{p}$ is a common starting point, not a theorem. In the benchmark, both versions use 100 trees, `max_depth=5`, `min_samples_leaf=2`, bootstrap samples, and `max_features=sqrt`. These restrictions keep the classroom run fast and make the scratch-versus-library comparison controlled; they are not claimed to be universally optimal.

7.6 Interpretation and common traps

A forest is less locally transparent than one tree because its prediction aggregates many paths. Mean decrease in impurity importance can favor variables with many possible split points and can divide credit unpredictably among correlated predictors. Permutation importance evaluates performance loss after shuffling a feature on held-out data, but correlated variables can substitute for one another and make each appear less important. Neither measure establishes causality.

Forests are usually robust tabular baselines, but they do not automatically solve probability calibration, class imbalance, missingness, dataset shift, or extrapolation beyond the observed feature range. Check

ranking, threshold metrics, probability loss, subgroup behavior, and stability rather than relying on accuracy or feature importance alone.

Matched seeds do not imply identical trees between scratch and scikit-learn. Libraries consume random numbers in different orders, break equal-gain ties differently, and optimize threshold enumeration. Table 3 shows that the matched forests have a test AUC gap of only -0.001 , with a paired interval containing zero, even though individual trees need not match. The proper verification targets are algorithmic invariants and controlled predictive behavior, not byte-for-byte tree identity.

8 AdaBoost: sequential focus on difficult rows

AdaBoost is easiest to understand as a *stagewise minimization of exponential loss*. Represent the labels and each weak classifier by $\tilde{y}_i, h_m(x_i) \in \{-1, +1\}$. After $m-1$ rounds, define the ensemble score

$$F_{m-1}(x) = \sum_{k=1}^{m-1} \alpha_k h_k(x).$$

The predicted class is $\text{sign}(F_{m-1}(x))$, while the magnitude $|F_{m-1}(x)|$ is a margin measuring confidence on the score scale.

8.1 Exponential loss and the weighted training sample

Consider the exponential empirical loss

$$\mathcal{L}(F) = \sum_{i=1}^n \exp\{-\tilde{y}_i F(x_i)\}.$$

At round m , AdaBoost adds one weak learner with coefficient α :

$$F_m(x) = F_{m-1}(x) + \alpha h_m(x).$$

Substituting this stagewise update into the loss gives

$$\begin{aligned} \mathcal{L}_m(\alpha) &= \sum_{i=1}^n \exp\{-\tilde{y}_i [F_{m-1}(x_i) + \alpha h_m(x_i)]\} \\ &= \sum_{i=1}^n \underbrace{\exp\{-\tilde{y}_i F_{m-1}(x_i)\}}_{w_i^{(m)}: \text{current difficulty weight}} \exp\{-\alpha \tilde{y}_i h_m(x_i)\}. \end{aligned}$$

Thus observations with small current margins $\tilde{y}_i F_{m-1}(x_i)$ receive relatively large weights. Define the unnormalized and normalized weights by

$$w_i^{(m)} = \exp\{-\tilde{y}_i F_{m-1}(x_i)\}, \quad D_i^{(m)} = \frac{w_i^{(m)}}{\sum_{k=1}^n w_k^{(m)}}, \quad \sum_i D_i^{(m)} = 1.$$

At the first round, $F_0 \equiv 0$, so $D_i^{(1)} = 1/n$. For a candidate stump h_m , its weighted error is

$$\epsilon_m = \sum_{i=1}^n D_i^{(m)} \mathbb{I}\{h_m(x_i) \neq \tilde{y}_i\} = \frac{\sum_i w_i^{(m)} \mathbb{I}\{h_m(x_i) \neq \tilde{y}_i\}}{\sum_i w_i^{(m)}}.$$

The weak learner is chosen to make ϵ_m as small as possible under the current weights.

8.2 Deriving the weak-learner coefficient

For fixed h_m , the only remaining scalar decision is α . Since $\tilde{y}_i h_m(x_i) = +1$ for a correct prediction and -1 for a mistake, split the weighted sum into correct and incorrect observations:

$$\begin{aligned} \mathcal{L}_m(\alpha) &\propto \sum_i D_i^{(m)} \exp\{-\alpha \tilde{y}_i h_m(x_i)\} \\ &= (1 - \epsilon_m) e^{-\alpha} + \epsilon_m e^{\alpha}. \end{aligned}$$

Differentiate with respect to α :

$$\frac{d\mathcal{L}_m}{d\alpha} = -(1 - \epsilon_m) e^{-\alpha} + \epsilon_m e^{\alpha}.$$

Setting the derivative to zero gives

$$\begin{aligned} \epsilon_m e^{\alpha_m} &= (1 - \epsilon_m) e^{-\alpha_m}, \\ e^{2\alpha_m} &= \frac{1 - \epsilon_m}{\epsilon_m}, \\ \alpha_m &= \boxed{\frac{1}{2} \log \frac{1 - \epsilon_m}{\epsilon_m}}. \end{aligned}$$

The second derivative is

$$(1 - \epsilon_m) e^{-\alpha} + \epsilon_m e^{\alpha} > 0,$$

so this stationary point is the minimizer. The formula also explains the behavior directly: if $\epsilon_m < 1/2$, then $\alpha_m > 0$; if $\epsilon_m = 1/2$, then $\alpha_m = 0$ and the learner contributes nothing. A classifier worse than random can be sign-flipped in the binary setting.

8.3 Deriving the observation-weight update

The next-round distribution is obtained from the new exponential-loss factors:

$$D_i^{(m+1)} = \frac{D_i^{(m)} \exp\{-\alpha_m \tilde{y}_i h_m(x_i)\}}{Z_m},$$

where Z_m normalizes the weights to sum to one. Hence

$$D_i^{(m+1)} \propto \begin{cases} D_i^{(m)} e^{-\alpha_m}, & h_m(x_i) = \tilde{y}_i, \\ D_i^{(m)} e^{\alpha_m}, & h_m(x_i) \neq \tilde{y}_i. \end{cases}$$

Correctly classified observations are downweighted and mistakes are upweighted. At the minimizing value of α_m ,

$$\begin{aligned} Z_m &= (1 - \epsilon_m) e^{-\alpha_m} + \epsilon_m e^{\alpha_m} \\ &= 2\sqrt{\epsilon_m(1 - \epsilon_m)}. \end{aligned}$$

Thus $Z_m < 1$ whenever the weak learner is better than random. This gives a compact mathematical explanation for why repeated weak improvements can reduce the exponential training objective.

The final ensemble score is

$$F_M(x) = \sum_{m=1}^M \alpha_m h_m(x), \quad \hat{y}(x) = \text{sign}(F_M(x)).$$

There is also a useful population interpretation. If $\pi(x) = \Pr(\tilde{Y} = +1 \mid X = x)$, then the conditional exponential risk is

$$\pi(x) e^{-F(x)} + [1 - \pi(x)] e^{F(x)}.$$

Its minimizer satisfies

$$F^*(x) = \frac{1}{2} \log \frac{\pi(x)}{1 - \pi(x)}, \quad \pi(x) = \sigma\{2F^*(x)\}.$$

This relation motivates a margin-to-probability transformation, but a finite AdaBoost fit is not automatically calibrated. Different library conventions can therefore preserve ranking and labels while producing noticeably different probabilities.

9 Gradient boosting: descent in function space

Gradient boosting replaces AdaBoost's fixed exponential loss with a more general principle: *take a gradient-descent step in the space of prediction functions*. Let

$$\mathcal{R}(F) = \frac{1}{n} \sum_{i=1}^n \ell(y_i, F(x_i))$$

be the empirical risk for a differentiable loss ℓ . At iteration m , the current function is F_{m-1} .

9.1 From ordinary gradient descent to a functional gradient

For an ordinary parameter vector θ , gradient descent uses $\theta \leftarrow \theta - \eta \nabla_{\theta} \mathcal{R}$. Here the trainable object is a function. At the observed inputs, the analogous negative-gradient values are

$$r_{im} = - \left. \frac{\partial \ell(y_i, F)}{\partial F} \right|_{F=F_{m-1}(x_i)}, \quad i = 1, \dots, n.$$

These r_{im} are called *pseudo-residuals*: they are residual-like quantities defined by the loss gradient, not necessarily by $y_i - F(x_i)$. Because the next step is restricted to a class of shallow regression trees $\mathcal{H}$, gradient boosting approximates this negative-gradient vector by solving

$$f_m \approx \arg \min_{f \in \mathcal{H}} \sum_{i=1}^n \{r_{im} - f(x_i)\}^2.$$

The update is then

$$F_m(x) = F_{m-1}(x) + \eta f_m(x),$$

where $0 < \eta \leq 1$ is the learning rate. The tree is therefore a structured approximation to the steepest-descent direction.

9.2 Binary cross-entropy pseudo-residuals

For binary classification with $y_i \in \{0, 1\}$, use the score-scale logistic loss

$$\ell(y_i, F_i) = \log(1 + e^{F_i}) - y_i F_i, \quad p_i = \sigma(F_i).$$

Differentiate step by step:

$$\begin{aligned} \frac{\partial \ell_i}{\partial F_i} &= \frac{e^{F_i}}{1 + e^{F_i}} - y_i \\ &= \sigma(F_i) - y_i \\ &= p_i - y_i. \end{aligned}$$

Hence the negative gradient is

$$r_{im} = y_i - p_i, \quad p_i = \sigma\{F_{m-1}(x_i)\}.$$

An observation with $y_i = 1$ but a small predicted probability has a large positive pseudo-residual, so the next tree tries to increase its score. An observation with $y_i = 0$ but a large predicted probability has a large negative pseudo-residual, so the next tree tries to decrease its score.

9.3 Deriving the initial constant score

Before fitting any tree, choose the best constant score

$$F_0(x) \equiv c^* = \arg \min_c \sum_{i=1}^n [\log(1 + e^c) - y_i c].$$

Let $\bar{y} = n^{-1} \sum_i y_i$. Differentiating gives

$$\begin{aligned}
0 &= \sum_{i=1}^n \{\sigma(c^*) - y_i\} \\
&= n\{\sigma(c^*) - \bar{y}\}.
\end{aligned}$$

Therefore $\sigma(c^*) = \bar{y}$, and

$$F_0 = \log \frac{\bar{y}}{1 - \bar{y}}.$$

Thus boosting starts from the training-set log-odds and lets subsequent trees model deviations from that constant baseline.

9.4 Why a regression-tree leaf stores a mean pseudo-residual

Suppose the fitted tree partitions the input space into terminal regions $R_{1m}, \dots, R_{J_m m}$. If a leaf R_{jm} stores a constant c_{jm} and the tree is fitted by least squares to the pseudo-residuals, then

$$c_{jm} = \arg \min_c \sum_{i: x_i \in R_{jm}} (r_{im} - c)^2.$$

Differentiate:

$$-2 \sum_{i: x_i \in R_{jm}} (r_{im} - c) = 0,$$

so

$$c_{jm} = \frac{1}{|R_{jm}|} \sum_{i: x_i \in R_{jm}} r_{im}.$$

This is the leaf rule used by the compact first-order implementation: each leaf stores the mean pseudo-residual, and

$$f_m(x) = \sum_{j=1}^{J_m} c_{jm} \mathbb{I}\{x \in R_{jm}\}.$$

9.5 Optional terminal-region optimization and the bridge to Newton boosting

After a tree structure has been chosen, a more aggressive implementation can optimize a separate score increment γ_{jm} inside each leaf:

$$\gamma_{jm} = \arg \min_{\gamma} \sum_{i: x_i \in R_{jm}} \ell(y_i, F_{m-1}(x_i) + \gamma).$$

For logistic loss this one-dimensional problem does not simplify to the mean pseudo-residual. A second-order approximation around $\gamma = 0$ gives

$$\ell_i(F_i + \gamma) \approx \ell_i(F_i) + g_i\gamma + \frac{1}{2}h_i\gamma^2,$$

with

$$g_i = p_i - y_i, \quad h_i = p_i(1 - p_i).$$

Within one leaf, differentiating the quadratic approximation yields

$$\begin{aligned} 0 &\approx \sum_{i \in R_{jm}} (g_i + h_i\gamma_{jm}), \\ \gamma_{jm} &\approx -\frac{\sum_{i \in R_{jm}} g_i}{\sum_{i \in R_{jm}} h_i} \\ &= \frac{\sum_{i \in R_{jm}} (y_i - p_i)}{\sum_{i \in R_{jm}} p_i(1 - p_i)}. \end{aligned}$$

This calculation is the direct bridge from first-order gradient boosting to the second-order, Newton-style leaf values used by XGBoost-style methods. It also explains why two implementations can fit very similar rankings while producing different probability scales: mean pseudo-residual leaves and optimized terminal values are different numerical updates.

10 XGBoost-style second-order trees

XGBoost-style boosting keeps the additive-tree structure of gradient boosting but uses a second-order approximation to the loss and an explicit tree regularizer. Let the current score be $F_{m-1}(x)$ and add a new tree f_m :

$$F_m(x) = F_{m-1}(x) + \eta f_m(x).$$

For deriving the tree itself, temporarily set aside the learning-rate factor η and optimize the unshrunk increment $f = f_m$.

10.1 Step 1: write the regularized stagewise objective

At iteration m , the part of the training objective involving the new tree is

$$\mathcal{J}^{(m)} = \sum_{i=1}^n \ell(y_i, F_{m-1}(x_i) + f(x_i)) + \Omega(f),$$

with tree regularization

$$\Omega(f) = \gamma T + \frac{\lambda}{2} \sum_{j=1}^T w_j^2.$$

Here T is the number of terminal leaves, w_j is the score stored in leaf j , $\gamma \geq 0$ penalizes adding leaves, and $\lambda \geq 0$ applies L2 shrinkage to leaf scores.

10.2 Step 2: take a second-order Taylor expansion

Write $F_i = F_{m-1}(x_i)$ and $f_i = f(x_i)$. Around the current score F_i ,

$$\ell(y_i, F_i + f_i) \approx \ell(y_i, F_i) + g_i f_i + \frac{1}{2} h_i f_i^2,$$

where

$$g_i = \left. \frac{\partial \ell(y_i, F)}{\partial F} \right|_{F=F_i}, \quad h_i = \left. \frac{\partial^2 \ell(y_i, F)}{\partial F^2} \right|_{F=F_i}.$$

For logistic loss,

$$\begin{aligned} \ell(y_i, F_i) &= \log(1 + e^{F_i}) - y_i F_i, \\ g_i &= \frac{e^{F_i}}{1 + e^{F_i}} - y_i = p_i - y_i, \\ h_i &= \frac{d}{dF_i} \sigma(F_i) = p_i(1 - p_i). \end{aligned}$$

The terms $\ell(y_i, F_i)$ do not depend on the new tree, so they can be dropped when comparing candidate trees. The approximate objective becomes

$$\tilde{\mathcal{J}}^{(m)}(f) = \sum_{i=1}^n \left[g_i f(x_i) + \frac{1}{2} h_i f(x_i)^2 \right] + \gamma T + \frac{\lambda}{2} \sum_{j=1}^T w_j^2.$$

10.3 Step 3: group observations by tree leaf

Let $q(x) \in \{1, \dots, T\}$ map an observation to its leaf and write

$$f(x) = w_{q(x)}.$$

For leaf j , define its index set and accumulated first- and second-order statistics

$$I_j = \{i : q(x_i) = j\}, \quad G_j = \sum_{i \in I_j} g_i, \quad H_j = \sum_{i \in I_j} h_i.$$

Then

$$\begin{aligned} \tilde{\mathcal{J}}^{(m)} &= \sum_{j=1}^T \left[\sum_{i \in I_j} g_i w_j + \frac{1}{2} \sum_{i \in I_j} h_i w_j^2 + \frac{\lambda}{2} w_j^2 \right] + \gamma T \\ &= \sum_{j=1}^T \left[G_j w_j + \frac{1}{2} (H_j + \lambda) w_j^2 \right] + \gamma T. \end{aligned}$$

Once the tree structure is fixed, the leaf weights separate: each w_j can be optimized independently.

10.4 Step 4: derive the optimal leaf value

For one leaf, minimize

$$\phi_j(w) = G_j w + \frac{1}{2}(H_j + \lambda)w^2.$$

Differentiate and set to zero:

$$\begin{aligned}\phi'_j(w) &= G_j + (H_j + \lambda)w = 0, \\ w_j^* &= \boxed{-\frac{G_j}{H_j + \lambda}}.\end{aligned}$$

The same result can be seen by completing the square:

$$\phi_j(w) = \frac{1}{2}(H_j + \lambda) \left(w + \frac{G_j}{H_j + \lambda} \right)^2 - \frac{1}{2} \frac{G_j^2}{H_j + \lambda}.$$

Therefore the optimized contribution of leaf j is

$$\phi_j(w_j^*) = -\frac{1}{2} \frac{G_j^2}{H_j + \lambda}.$$

For a fixed tree structure q , the optimized approximate objective is thus

$$\boxed{\tilde{\mathcal{J}}^{(m)}(q) = -\frac{1}{2} \sum_{j=1}^T \frac{G_j^2}{H_j + \lambda} + \gamma T}$$

up to the dropped constant from the previous ensemble.

10.5 Step 5: derive the split-gain formula

Suppose a parent leaf with statistics (G_P, H_P) is split into left and right children. Since the children partition the parent,

$$G_P = G_L + G_R, \quad H_P = H_L + H_R.$$

Before the split, the optimized parent contribution is

$$-\frac{1}{2} \frac{G_P^2}{H_P + \lambda} + \gamma.$$

After the split, the two optimized child contributions are

$$-\frac{1}{2} \frac{G_L^2}{H_L + \lambda} - \frac{1}{2} \frac{G_R^2}{H_R + \lambda} + 2\gamma.$$

Define gain as “objective before minus objective after.” Then

$$\begin{aligned} \text{Gain} &= \frac{1}{2} \left[\frac{G_L^2}{H_L + \lambda} + \frac{G_R^2}{H_R + \lambda} - \frac{G_P^2}{H_P + \lambda} \right] - \gamma \\ &= \frac{1}{2} \left[\frac{G_L^2}{H_L + \lambda} + \frac{G_R^2}{H_R + \lambda} - \frac{(G_L + G_R)^2}{H_L + H_R + \lambda} \right] - \gamma. \end{aligned}$$

A split is useful only when the reduction in the quadratic loss approximation is large enough to pay the extra leaf penalty γ .

10.6 Interpretation and relation to Newton’s method

The gradient sum G_j determines the direction in which a leaf score should move. The Hessian sum H_j measures local curvature: when curvature is large, a given gradient produces a smaller Newton step. The regularizer λ further shrinks the leaf value and prevents unstable updates when H_j is small.

If $\lambda = 0$, then

$$w_j^* = -\frac{G_j}{H_j}$$

is exactly the Newton step for a constant score increment within that leaf. XGBoost-style training can therefore be viewed as repeatedly constructing a tree-structured approximation to a Newton step, followed by shrinkage through the learning rate η . Minimum child weight, row subsampling, and column subsampling provide additional regularization.

The scratch version enumerates exact observed thresholds. Production XGBoost uses a histogram engine in the matched reference configuration and adds missing value logic, parallelism, memory optimizations, and numerical safeguards [6].

11 LightGBM-style histograms and leaf-wise growth

Exact split search repeatedly considers many observed feature values. Histogram training first maps continuous values into a fixed number of ordered bins. It then accumulates gradient and Hessian sums per bin and evaluates only bin boundaries. With $B \ll n$ bins, split search can be much cheaper and cache friendlier, at the price of quantizing thresholds.

Depth-wise growth expands nodes level by level. Leaf-wise growth maintains all current leaves, evaluates their best available splits, and expands the globally highest-gain leaf. This can reduce training loss quickly for a fixed number of leaves, but on a small dataset it can create an uneven, highly specialized branch. The scratch class therefore exposes both `num_leaves` and `max_depth`.

The classroom implementation captures quantile bins, gradient/Hessian histograms, and best-leaf-first growth. It does not reproduce distributed training, exclusive feature bundling, gradient-based one-side sampling, categorical-feature algorithms, or every production missing-value behavior described by Ke et al. [7].

12 Measured scratch-versus-library results

12.1 Held-out ranking consistency

All eight pre-declared pairs meet the AUC consistency rule. Table 3 reports the untouched 114-row test set. The largest observed absolute gap is 0.006 for unregularized logistic regression; its interval includes zero.

Family	Scratch AUC	Library AUC	Gap	95% paired interval
Logistic none	0.984	0.977	+0.006	$[-0.002, 0.024]$
Logistic L1	0.990	0.991	-0.001	$[-0.004, 0.000]$
Logistic L2	0.989	0.989	0.000	$[0.000, 0.000]$
Random forest	0.991	0.992	-0.001	$[-0.006, 0.002]$
AdaBoost	0.993	0.993	0.000	$[0.000, 0.000]$
Gradient boosting	0.992	0.994	-0.002	$[-0.009, 0.005]$
XGBoost style	0.993	0.991	+0.002	$[-0.000, 0.007]$
LightGBM style	0.993	0.994	-0.002	$[-0.006, 0.001]$

Table 3: Scratch minus library ROC AUC under the fixed paired protocol.

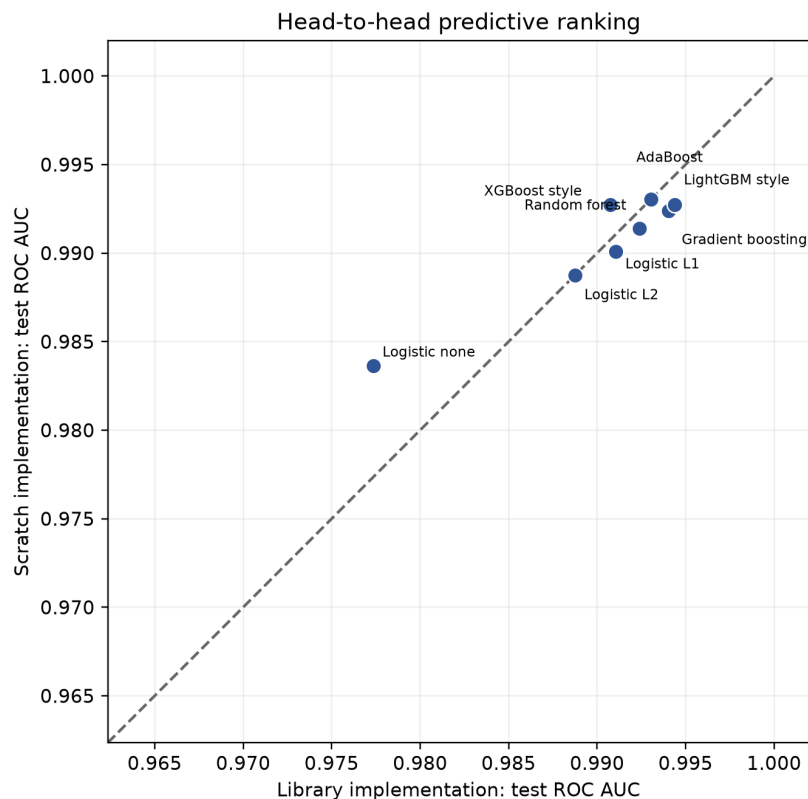


Figure 4: Points near the diagonal show held-out ranking consistency. The plot does not establish calibration equivalence.

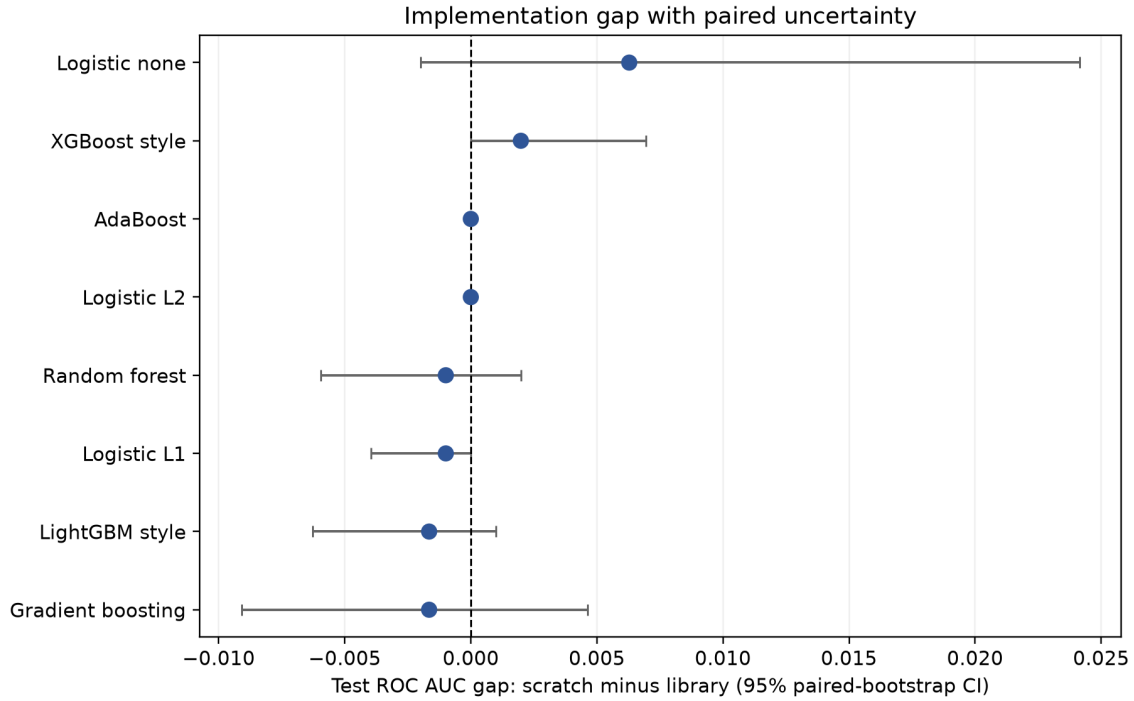


Figure 5: Class-stratified paired-bootstrap intervals quantify uncertainty in each scratch-minus-library AUC gap.

12.2 Probability discrepancies are the main teaching result

Family	Label agreement	Probability MAE	Scratch log loss	Library log loss
Logistic none	0.982	0.014	0.725	0.634
Logistic L1	1.000	0.005	0.095	0.088
Logistic L2	1.000	$< 10^{-6}$	0.099	0.099
Random forest	0.991	0.015	0.107	0.101
AdaBoost	1.000	0.293	0.118	0.375
Gradient boosting	0.956	0.138	0.217	0.082
XGBoost style	0.991	0.006	0.087	0.094
LightGBM style	0.982	0.012	0.090	0.095

Table 4: Agreement in ranking or labels need not imply agreement in probability scale.

AdaBoost is the cleanest example: AUC and thresholded labels agree exactly, yet probability MAE is 0.293. The versions use different margin-to-probability conventions. This supports ranking consistency and label consistency under the chosen threshold, but not probability equivalence.

Gradient boosting has a smaller AUC gap but a probability MAE of 0.138. Its mechanism is different: the scratch version stores raw pseudo-residual means, whereas the library optimizes terminal-region values. Here the data support similar ranking but materially different confidence.

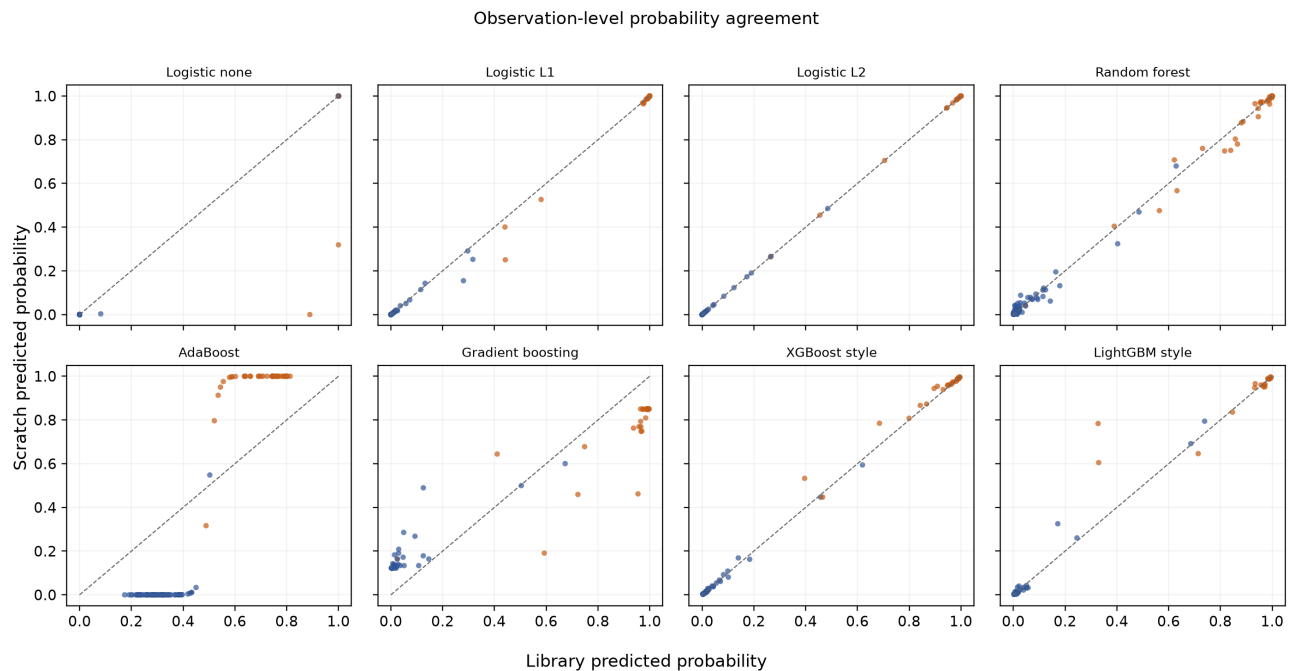


Figure 6: Observation-level probability comparisons reveal transformation and terminal-value differences that AUC hides.

12.3 The unregularized separation warning

The 455-row standardized training sample is separable. Unregularized logistic cross-entropy therefore has an infimum of zero but no finite coefficient minimizer: scaling a separating direction makes the training loss progressively smaller. The scratch method reaches its iteration budget rather than satisfying a finite-parameter convergence rule. The library also produces very large scores under its solver rules.

Ranking remains strong, but a few confident test mistakes yield log losses of 0.725 and 0.634. L1 and L2 fits have test log losses below 0.10. The lesson is not that logistic regression failed; it is that unpenalized maximum likelihood is ill-posed under separation and ranking can conceal extreme probabilities.

12.4 Parameter verification

The matched L2 objectives agree to displayed numerical precision, test probabilities differ by less than 10^{-6} on average, and coefficient correlation exceeds 0.98 in the unit test. L1 is more solver-sensitive near the selection boundary: standardized coefficient correlation is 0.835, mean absolute coefficient difference is 0.178, and support Jaccard similarity is 0.938. These results are consistent with matching the same nonsmooth objective up to solver tolerance while some near-zero coefficients remain unstable.

13 How to interpret model choice responsibly

The fixed results do not justify selecting the row with the highest AUC. The test set has only 42 positive and 72 negative observations, so apparent differences of a few thousandths are not stable evidence of broad superiority. The following questions are more useful:

1. Is the target scientifically and operationally meaningful?

2. Which errors matter, and does threshold 0.5 reflect those costs?
3. Are calibrated probabilities needed, or is ranking enough?
4. Does the new population match the training population and measurement process?
5. Is a compact linear model adequate, or do nonlinear interactions have a credible role?
6. Can the pipeline be reproduced, monitored, and independently validated?

For this benchmark, regularized logistic regression is a strong default: it is compact, stable, and almost indistinguishable from the library reference. Tree ensembles provide a useful nonlinear comparison. Any deployment would still need calibration assessment, threshold selection under explicit costs, external validation, drift monitoring, and human review. The dataset is a classroom benchmark, not a clinical validation study.

14 Common failure modes and diagnostic questions

Symptom	Questions to ask
Loss becomes NaN	Was sigmoid or cross-entropy implemented stably? Are inputs finite? Is the step too large?
Scratch L2 differs strongly	Are features standardized identically? Is $\lambda = 1/(nC)$ under the chosen conventions? Is the intercept excluded?
L1 support differs	Are near-zero tolerances comparable? Did both solvers converge? Is the objective scaled identically?
Forest trees differ	Are predictive metrics and invariants stable despite different random streams and tie-breaking?
AUC agrees, log loss differs	Is the margin-to-probability mapping different? Are terminal leaf values optimized differently?
Training metric is perfect	Was the test set untouched? Is the model separated or overfit? What do probability metrics show?
Fast library, slow scratch	Which production optimizations are absent, and does runtime affect the mathematical verification claim?

15 Extensions

Natural extensions include multinomial logistic regression, class/sample weights, missing-data policies, native categorical splits, monotonic constraints, calibration curves, threshold selection, partial dependence, permutation importance, SHAP-style explanations, nested cross-validation, and external validation. These should be introduced as new lessons with explicit assumptions rather than quietly folded into the core classes.

For a structured-data sequel, students can compare one-hot encoded linear models with native categorical boosting, or study grouped and temporal splits where row-wise random splitting would leak information. The same verification principle applies: expose the governing mechanism, control the comparison, and diagnose the dimensions of agreement separately.

A Pseudocode

A.1 Accelerated proximal logistic training

1. Form $\tilde{X} = [\mathbf{1}, X]$ and initialize $w = 0$, extrapolated point $v = w$, and momentum $t = 1$.
2. Compute the smooth gradient at v .
3. Take $u = v - \eta \nabla \ell(v)$, including the L2 gradient when relevant.
4. For L1, soft-threshold coefficient coordinates of u ; never threshold the intercept.
5. If the objective increased, restart at w and backtrack η .
6. Record data loss, penalty, and total objective.
7. Stop when $\|u - w\| \leq \text{tol}(1 + \|w\|)$; otherwise form the accelerated extrapolation and continue.

A.2 Greedy CART classification tree

1. At node $\mathcal{N}$, compute its weighted positive fraction and Gini impurity; store the fraction as the node's fallback prediction.
2. If the node is pure or violates a depth or sample-size stopping rule, return it as a leaf.
3. Use all features for an ordinary CART tree, or draw the allowed candidate feature set when `max_features` requests random feature subsampling.
4. For each feature, sort its node values and scan midpoints between adjacent distinct values using cumulative class weights.
5. Choose the valid split with largest weighted Gini decrease.
6. If no split clears `min_impurity_decrease`, return a leaf.
7. Otherwise partition the row indices and apply these steps recursively to the left and right children.

At prediction time, follow one branch at each internal node and return the positive fraction stored in the reached leaf.

A.3 Random forest

1. For $m = 1, \dots, M$, sample training row indices with replacement.
2. Grow a CART tree; at every node sample a fresh feature subset.
3. Store the weighted positive proportion in each leaf.
4. Average the M positive-class probabilities at prediction time.
5. Optionally, for each training row, average only trees for which that row was out of bag to obtain an internal diagnostic prediction.

A.4 Newton boosting

1. Initialize the score with the training log-odds.
2. At round m , compute $p = \sigma(F)$, $g = p - y$, and $h = p(1 - p)$.
3. Optionally subsample rows and columns.
4. Grow a tree using regularized second-order split gain.
5. Store $-G/(H + \lambda)$ in each leaf.
6. Update $F \leftarrow F + \eta f_m$.

B Reproducibility commands

```
uv sync --locked --extra tabular
DYLD_LIBRARY_PATH=.local/libomp/18.1.5/lib make benchmark-tabular
DYLD_LIBRARY_PATH=.local/libomp/18.1.5/lib make validate-homework-tabular
make notes-tabular
make homework-tabular
make verify-slides-tabular
make test
```

The explicit OpenMP path is needed only on a compatible Apple Silicon setup where XGBoost/LightGBM cannot find the runtime automatically. A normal system installation can use its own documented path.

References

- [1] W. Wolberg, O. Mangasarian, N. Street, and W. Street. *Breast Cancer Wisconsin (Diagnostic)*. UCI Machine Learning Repository, DOI: [10.24432/C5DW2B](https://doi.org/10.24432/C5DW2B).
- [2] L. Breiman, J. Friedman, R. Olshen, and C. Stone. *Classification and Regression Trees*. Wadsworth, 1984.
- [3] L. Breiman. Random Forests. *Machine Learning*, 45:5–32, 2001. DOI: [10.1023/A:1010933404324](https://doi.org/10.1023/A:1010933404324).
- [4] Y. Freund and R. Schapire. A Decision-Theoretic Generalization of On-Line Learning and an Application to Boosting. *Journal of Computer and System Sciences*, 55(1):119–139, 1997.
- [5] J. Friedman. Greedy Function Approximation: A Gradient Boosting Machine. *Annals of Statistics*, 29(5):1189–1232, 2001.
- [6] T. Chen and C. Guestrin. XGBoost: A Scalable Tree Boosting System. *Proceedings of KDD*, 2016. DOI: [10.1145/2939672.2939785](https://doi.org/10.1145/2939672.2939785).
- [7] G. Ke et al. LightGBM: A Highly Efficient Gradient Boosting Decision Tree. *Advances in Neural Information Processing Systems*, 2017.